Spectral Word Embeddings

Outline

- Why represent words as vectors?
- The distributional hypothesis
- Measuring word similarity from context
- The word similarity graph
- The graph Laplacian
- Spectral embedding algorithm
- Properties and applications
- Connection to other methods

Part I: Why Embeddings?

The Problem with Discrete Words

In n-gram models, words are atomic symbols — just indices

"cat" is index 3,417; "dog" is index 8,902

No notion of similarity: **every pair of words is equally different**

One-Hot Encoding

Represent each word as a vector in $\mathbb{R}^{|V|}$ with one nonzero entry:

$$\text{cat} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad \text{dog} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

All pairs are orthogonal: $\langle \mathbf{e}_i, \mathbf{e}_j \rangle = 0$

Problems with One-Hot

- Every word equidistant from every other
- "cat"–"dog" same distance as "cat"–"telescope"
- 50,000 words → 50,000-dimensional vectors
- 99.998% zeros — extremely sparse

The geometry carries **no linguistic information**

Word Embeddings

Map each word to a dense vector in $\mathbb{R}^d$ where $d \ll |V|$:

$$\phi : V \to \mathbb{R}^d$$

Constructed so geometry reflects linguistic structure:

- Similar words nearby
- Dissimilar words far apart
- Relationships encoded as directions

What Embeddings Buy Us

Generalization: knowledge about "cat" transfers to "dog"

Dimensionality reduction: $d \approx$ 100-300 instead of $|V|$

Geometric structure: distance, angle, clustering, visualization

ML input: continuous vectors for classifiers, neural networks

Part II: The Distributional Hypothesis

Firth (1957)

> *"You shall know a word by the company it keeps."*

Words in similar **contexts** tend to have similar **meanings**

"cat" and "dog" appear near the same words: "the," "sat," "pet," ...

Connection to N-gram Models

N-gram models are fundamentally about **context**

- They estimate $P(w \mid \text{context})$ from co-occurrence counts
- The distributional hypothesis: these same co-occurrence patterns determine word similarity

We already have the raw material — now we extract structure from it

Defining Context

For a target word w, define positional features:

- w_i = word at relative position i

Example: "Computer Science is no more about **computers** than astronomy"

- $w_1 = \text{than}$
- $w_{-1} = \text{about}$
- $w_{-2} = \text{more}$

Paired Context Features

Tuple features $w_{i,j}$ capture word pairs at positions i and j:

- $w_{-4,2} = (\text{is}, \text{astronomy})$

Richer patterns: encode which words co-occur in specific configurations

The Full Context Set

$$C(w) = \{w_i \mid i \in \mathbb{Z},\, i \neq 0\} \cup \{w_{i,j} \mid i, j \in \mathbb{Z},\, i \neq j\}$$

In practice: restrict to a fixed window, aggregate across all occurrences in corpus

Part III: Measuring Similarity

The Dice Coefficient

Compare two words by the overlap of their context sets:

$$D(w^{(i)}, w^{(j)}) = 2 \times \frac{|C(w^{(i)}) \cap C(w^{(j)})|}{|C(w^{(i)})| + |C(w^{(j)})|}$$

- $D = 1$: identical contexts
- $D = 0$: no shared contexts
- Symmetric, bounded in $[0, 1]$

The Similarity Matrix

Compute $D(w^{(i)}, w^{(j)})$ for every pair $\rightarrow$ matrix $W \in \mathbb{R}^{|V| \times |V|}$

$$W_{ij} = D(w^{(i)}, w^{(j)})$$

- **Symmetric**: $W_{ij} = W_{ji}$
- **Non-negative**: $W_{ij} \geq 0$
- **Diagonal = 1**: $W_{ii} = 1$

Example: Similarity Table

	$w^{(1)}$	$w^{(2)}$	$w^{(3)}$	...
$w^{(1)}$	1	D_{12}	D_{13}	...
$w^{(2)}$	D_{21}	1	D_{23}	...
$w^{(3)}$	D_{31}	D_{32}	1	...

For $|V| = 50{,}000$: 2.5 billion entries

Need a compact, low-dimensional representation

Part IV: The Word Graph

From Matrix to Graph

W is the **adjacency matrix** of a weighted graph $G = (V, E)$:

- **Vertices**: words
- **Edges**: between every pair, weight $= W_{ij}$

Strong edges → similar words; weak edges → dissimilar

What the Graph Captures

Words sharing many contexts form tightly knit clusters

- Nouns cluster with nouns
- Verbs cluster with verbs
- Finer structure within: animals together, cooking verbs together

The topology encodes language structure from the corpus

Part V: The Graph Laplacian

The Degree Matrix

Degree of vertex i: sum of all incident edge weights

$$d_i = \sum_{j=1}^{|V|} W_{ij}$$

Degree matrix: $D = \text{diag}(d_1, d_2, \ldots, d_{|V|})$

High degree $\rightarrow$ common word with diverse usage

Definition

The **(unnormalized) graph Laplacian**:

$$L = D - W$$

Entries:

$$L_{ij} = \begin{cases} d_i & \text{if } i = j \\ -W_{ij} & \text{if } i \neq j \end{cases}$$

The Key Identity

For any vector $\mathbf{f} \in \mathbb{R}^{|V|}$:

$$\mathbf{f}^\top L \mathbf{f} = \frac{1}{2} \sum_{i,j} W_{ij} (f_i - f_j)^2$$

Measures total **variation** of $\mathbf{f}$ over the graph

Interpreting the Quadratic Form

- Small $\mathbf{f}^\top L\mathbf{f}$: similar values at strongly connected vertices
- Large $\mathbf{f}^\top L\mathbf{f}$: different values at strongly connected vertices

Finding $\mathbf{f}$ that minimizes this = assigning coordinates that keep similar words close

Positive Semi-Definiteness

Since $W_{ij} \geq 0$ and $(f_i - f_j)^2 \geq 0$:

$$\mathbf{f}^\top L \mathbf{f} \geq 0 \quad \text{for all } \mathbf{f}$$

All eigenvalues non-negative: $0 = \lambda_1 \leq \lambda_2 \leq \cdots$

The Zero Eigenvalue

The constant vector $\mathbf{1}$ is always an eigenvector with eigenvalue 0:

$$L\mathbf{1} = (D - W)\mathbf{1} = D\mathbf{1} - W\mathbf{1} = \mathbf{0}$$

A constant function has **zero variation —** makes intuitive sense

The Normalized Laplacian

Random walk normalization:

$$L_{\text{rw}} = D^{-1}L = I - D^{-1}W$$

$D^{-1}W$ is the **transition matrix** of a random walk on the graph

Normalization corrects for word frequency bias

Part VI: Spectral Embedding

The Optimization Problem

Find $f : V \to \mathbb{R}$ minimizing variation, excluding trivial solutions:

$$\min_{\mathbf{f}} \ \mathbf{f}^\top L\mathbf{f} \quad \text{s.t.} \quad \mathbf{f}^\top D\mathbf{f} = 1, \quad \mathbf{f}^\top D\mathbf{1} = 0$$

Generalized Eigenvalue Problem

By Lagrange multipliers, the solution satisfies:

$$L\mathbf{f} = \lambda D\mathbf{f}$$

Equivalently: eigenvectors of $D^{-1}L = I - D^{-1}W$

The k smallest nonzero eigenvalues give the best k-dimensional embedding

The Fiedler Vector

$\mathbf{f}_2$ (smallest nonzero eigenvalue λ_2) = the **Fiedler vector**

Best 1D embedding of the graph

Tends to split the graph into its two most loosely connected parts

Building the k-Dimensional Embedding

Take eigenvectors $\mathbf{f}_2, \mathbf{f}_3, \ldots, \mathbf{f}_{k+1}$ as columns:

$$\Phi = \begin{bmatrix} \mathbf{f}_2 & \mathbf{f}_3 & \cdots & \mathbf{f}_{k+1} \end{bmatrix} \in \mathbb{R}^{|V| \times k}$$

Embedding of word i = row i of Φ:

$$\phi(w^{(i)}) = (f_2(i), f_3(i), \ldots, f_{k+1}(i)) \in \mathbb{R}^k$$

The Algorithm

1. Compute context sets $C(w)$ for each word
2. Build similarity matrix W (Dice coefficient)
3. Compute degree matrix D
4. Compute Laplacian $L = D - W$
5. Solve $L\mathbf{f} = \lambda D\mathbf{f}$ for smallest eigenvalues
6. Form Φ from eigenvectors of smallest nonzero λ's
7. Read off word vectors as rows of Φ

Why It Works: Intuition

Eigenvectors = **natural modes of vibration** of a spring network

- Strong edges = stiff springs
- Lowest-frequency modes = broadest structure
- Higher modes = finer distinctions

Using k lowest modes captures k most important axes of variation

Part VII: Properties and Applications

Optimality Guarantee

Spectral embedding minimizes:

$$\sum_{i,j} W_{ij} \|\phi(w^{(i)}) - \phi(w^{(j)})\|^2$$

Total "stretching" across strong edges is minimized

Similar words → nearby vectors

Finding Similar Words

Given word w, find nearest neighbors:

$$\text{NN}_m(w) = m \text{ closest vectors to } \phi(w)$$

Compute Euclidean distances to all other word vectors

Example: Neighbors of "made"

Query "made" → neighbors might include:

built, created, produced, constructed, formed, designed, developed

Cluster of past-tense creation verbs — emerges purely from context statistics

Discovering Structure

- Past-tense forms cluster with past-tense forms
- Plural nouns cluster with plural nouns
- Grammatical categories emerge from distributional statistics

No linguistic knowledge was given — structure is **data-driven**

Clustering

Word vectors as input to clustering algorithms (k-means, etc.)
Clusters correspond to densely connected graph regions
Data-driven word categorization without hand-built lexicons

Visualization

Low-dimensional embeddings ($k = 2$ or $k = 3$) $\rightarrow$ scatter plots

A "megascope" for exploring large amounts of linguistic data:

- Compare word neighborhoods
- Study differences between corpora
- Compare languages side-by-side

Part VIII: Connections

Count-Based Methods

Spectral embedding belongs to the **count-based** family:

LSA: word-context matrix → SVD → low-rank approximation

PMI methods: replace counts with pointwise mutual information

GloVe: factorizes log-count co-occurrence matrix via weighted least squares

All derive vectors from co-occurrence statistics

Neural Methods

Word2Vec, **ELMo**, **fastText** (2013-2018):

- Same distributional principle
- Learn embeddings via neural network training
- ELMo: contextualized embeddings from deep bidirectional LSTM

Different algorithm, same foundation — covered in future articles

What Spectral Embedding Offers

Mathematical transparency: build matrix, compute eigenvalues — no training loops
Theoretical guarantees: provably minimizes graph cut objective
Interpretability: each dimension = specific eigenvector
Limitation: forming and decomposing $|V| \times |V|$ matrix is expensive

Summary

- **One-hot** vectors carry no similarity information
- **Distributional hypothesis**: context determines meaning
- **Dice coefficient** quantifies context overlap
- **Similarity matrix** W defines a weighted word graph
- **Graph Laplacian** $L = D - W$ measures variation
- **Key identity**: $\mathbf{f}^\top L \mathbf{f} = \frac{1}{2} \sum_{i,j} W_{ij} (f_i - f_j)^2$
- **Spectral embedding**: eigenvectors of $L\mathbf{f} = \lambda D\mathbf{f}$
- Similar words $\rightarrow$ nearby vectors $\rightarrow$ geometry encodes linguistics